PesaFlow

SOFTWARE PROJECT BLUEPRINT

M-Pesa Finance Manager

A personal finance platform for the M-Pesa generation — from a solo-built MVP to a production-ready SaaS product.

v1 · MVP scope defined | 4-phase roadmap | Architecture & ERD included

PREPARED FOR
Ade — Founder & Lead Engineer

DOCUMENT TYPE
Master Product & Engineering Blueprint

VERSION
1.0 · Draft for Build

DOCUMENT MAP

Table of Contents

Twenty sections, organized the way a product team would sequence discovery, design, and delivery planning.

How to use this document

This blueprint is written as a planning artifact, not a tutorial. It defines **what** to build, **why**, **for whom**, and **in what order** — deliberately scoped around your current stack (FastAPI, SQLAlchemy, Alembic, Pydantic, Jinja2, vanilla JS, SQLite) so Version 1 is realistic to ship solo, while every architectural decision leaves a clear path toward PostgreSQL, background workers, mobile clients, and AI-powered insights later.

Treat Sections 01–04 as the "why," Sections 05–13 as the "what/how it looks," and Sections 14–20 as the "how we make it real and keep it healthy."

SECTION 01

Executive Summary & Vision

What PesaFlow is, why it matters, and where it's ultimately headed.

PesaFlow is a personal finance web application built around the financial reality of most Kenyans and East Africans: money that moves primarily through **M-Pesa**, not bank accounts. Instead of forcing users into generic budgeting apps designed for card-and-bank economies, PesaFlow starts from M-Pesa statements and transaction patterns, and grows into a full financial management platform — budgeting, reporting, analytics, savings goals, and eventually AI-driven financial insight.

The long-term vision is a production-grade SaaS product. The near-term goal is a portfolio-defining V1 that is **realistic to build solo** with your current stack, while every architectural choice below is made so that V1 doesn't have to be thrown away — it gets extended, not replaced.

THE CORE IDEA

Turn a messy M-Pesa statement into a clean, categorized, understandable picture of a person's financial life — automatically.

THE WEDGE

No mainstream budgeting tool treats M-Pesa as a first-class data source. PesaFlow does, from day one.

THE TRAJECTORY

Tracker → Budgeter → Analytics platform → AI financial co-pilot, released in that order, never all at once.

Guiding Principles

PRINCIPLE	WHAT IT MEANS IN PRACTICE
Scope discipline	V1 ships a small, complete, useful product. Ambitious ideas are recorded in the roadmap, not squeezed into the MVP.
Build on what you know	Every V1 requirement maps to a skill you already have. Nothing in V1 requires a new language, framework, or paradigm.
Design for the next version	Data models and API boundaries are shaped so that PostgreSQL, background workers, and multi-account support slot in without a rewrite.
Financial data is sensitive by default	Every design decision treats transaction data as sensitive personal data, even in V1, even with one user.
Portfolio-grade polish	The UI, README, and demo experience are treated as product surfaces — this is the project that represents you.

SECTION 02

Problem Statement & Opportunity

Why existing tools fail M-Pesa users, and where the opening is.

The problem

M-Pesa is the primary financial rail for tens of millions of people, handling salary payments, merchant purchases (Till/Paybill), person-to-person transfers, airtime, savings (M-Shwari/KCB-M-Pesa), and bill payments. Yet the standard way people understand their own spending is scrolling through raw M-Pesa SMS messages or a dense, unformatted PDF/CSV statement.

WHERE USERS ARE TODAY

- M-Pesa statements are exported as dense PDFs or CSVs with cryptic codes and no categorization.
- Generic budgeting apps (Mint-style, card-first) assume bank/card rails that don't reflect local usage.
- Spreadsheets require manual discipline most people abandon within weeks.
- There is no simple way to answer "where did my money actually go this month?" in under a minute.

WHAT THAT COSTS PEOPLE

- Spending blind spots — Fuliza/overdraft usage, small recurring Till payments, airtime creep.
- No feedback loop between "I want to save" and actual month-to-month behavior.
- Financial stress from lack of visibility rather than lack of income.
- Community groups (chamas, church funds) tracked in notebooks or ad-hoc spreadsheets.

The opportunity

A tool that treats the **M-Pesa statement as the primary data source** — parsing it automatically, categorizing transactions intelligently, and presenting a clear dashboard — removes the single biggest piece of friction in personal finance for this market: data entry. Everything else (budgets, goals, analytics, AI insight) compounds in value once that friction is gone.

WHY NOW, WHY YOU

You already sit at the intersection of the skills this needs (FastAPI backend, structured data modeling, front-end fundamentals) and the lived context that makes the product real (active M-Pesa user, forex trader who already thinks in flows of money, community ties through church groups that could become early testers). Few builders have both the technical grounding and the domain instinct at the same time.

SECTION 03

Product Overview & Target Users

One-line pitch, product pillars, and the people PesaFlow is designed around.

ONE-LINE PITCH

"PesaFlow turns your M-Pesa statement into a clear, categorized picture of your money — and helps you do something about it."

Product pillars

01 · VISIBILITY

Import and auto-categorize M-Pesa activity so users see their real spending without manual entry.

02 · CONTROL

Budgets, category limits, and savings goals turn visibility into intentional decisions.

03 · INSIGHT

Reports and, later, AI-driven analysis surface patterns a person wouldn't notice on their own.

Primary personas

PERSONA	CONTEXT	WHAT THEY NEED FROM PESAFLOW
Amina, 26 Salaried professional	Receives salary via bank but spends almost entirely through M-Pesa: rent via Paybill, transport, groceries, airtime, chama contributions.	Wants a monthly breakdown of where her salary actually goes and a nudge before she overspends on transport/food.
Brian, 31 Small business owner	Runs a small retail stall with Till Number payments mixed with personal spending on the same phone number.	Needs to separate business inflow from personal expenses and see net cash position at a glance.
Grace, 34 Church group treasurer	Manages contributions and disbursements for a church fund or chama, currently tracked in a notebook.	(V3+) Wants shared visibility and simple reporting for group funds — validates your church-tech background.
You (the builder) Primary user & tester	Active M-Pesa user and forex trader who already thinks in terms of inflows, outflows, and disciplined tracking.	A daily-use tool honest enough about its own limitations to trust with real financial data — the best possible product feedback loop.

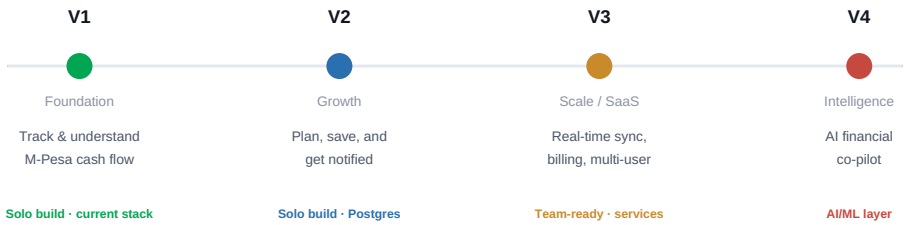
Positioning note

V1 targets Amina and you — the individual user managing personal M-Pesa cash flow. Brian's business/personal separation is a natural V2 extension (multi-account support). Grace's group-fund use case is intentionally deferred to V3, where multi-tenant/shared-access architecture exists to support it properly.

SECTION 04

Release Roadmap — V1 → V4

Four deliberate phases. Each one is a shippable, demoable product on its own.



Phase-by-phase scope

PHASE	HEADLINE CAPABILITIES	NEW TECH INTRODUCED	GOAL
V1	Auth, manual transactions, M-Pesa statement import & categorization, budgets, dashboard, reports, CSV export	None — pure execution on current stack	A working, demoable, portfolio-ready product
V2	PostgreSQL migration, savings goals, recurring-transaction detection, alerts/notifications, multi-account support, PWA	PostgreSQL, background jobs (APScheduler/Celery), email service	A tool you and early testers use every week
V3	Daraja (M-Pesa) API integration, subscription billing, multi-tenant accounts, Telegram bot, public API, shared/group budgets	Redis, message queue, Docker, payment gateway, Telegram Bot API	A real SaaS with paying users
V4	AI chat over personal transaction history (RAG), spend forecasting, anomaly detection, auto-categorization via ML	LangChain/RAG, vector DB (pgvector), ML classifier service	A financial co-pilot, not just a tracker

SEQUENCING RULE

Never start a phase before the previous one has real users (even if that "user" is only you, using it daily for a full month). Each phase should be justified by a concrete pain point the previous phase exposed — not by excitement about a new technology.

SECTION 05

Version 1 Feature Breakdown

Everything V1 includes — and, just as importantly, what it deliberately excludes.

MODULE	V1 SCOPE
Authentication	Email + password registration/login, JWT access tokens, password hashing (bcrypt/argon2), session expiry & refresh, basic "forgot password" flow.
Manual transactions	Create/edit/delete income & expense entries with amount, category, date, note. Full CRUD with pagination, search, and filtering by date range/category/type.
M-Pesa statement import	Upload M-Pesa PDF/CSV statement → parse transactions → auto-suggest categories via rule-based matching → preview & confirm screen → bulk insert. Duplicate detection via M-Pesa transaction code.
Categories	System default categories (Salary, Transport, Airtime, Food, Bills, Transfers, Business, Shopping, Savings, Other) plus user-defined custom categories with icon/color.
Budgets	Monthly spending limit per category, progress bar, over-budget indicator. No rollover logic yet — reset each calendar month.
Dashboard	Current balance, this-month income vs. expense, top spending categories, recent transactions, budget status at a glance.
Reports	Monthly summary, category breakdown (chart), income vs. expense trend across recent months.
Data export	Export transactions to CSV for a selected date range.
Account & settings	Profile edit, password change, light/dark theme toggle, data reset (danger zone).

IN SCOPE FOR V1

- Single default "M-Pesa account" per user (no multi-account switching yet)
- Rule-based categorization (keyword/merchant matching), not ML
- SQLite as the only database
- Server-rendered Jinja2 pages + vanilla JS for interactivity
- CSV/PDF statement import only (no live API sync)

EXPLICITLY OUT OF SCOPE FOR V1

- PostgreSQL, Docker, background job queues
- Multi-account / multi-user / shared budgets
- Live Daraja API integration
- Notifications, email alerts, mobile app, Telegram bot
- AI/ML categorization or financial insight

DEFINITION OF "DONE" FOR V1

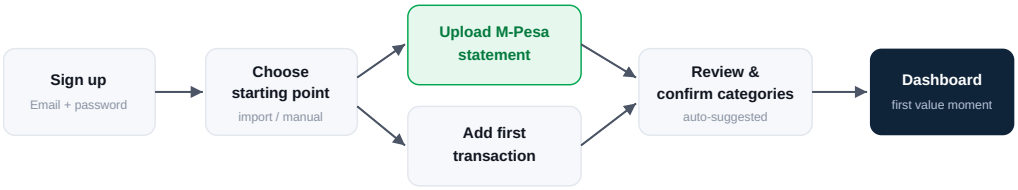
A user can register, upload a real M-Pesa statement, see it parsed into categorized transactions within seconds, set a budget for at least one category, and open the dashboard the next day to see an accurate, trustworthy picture of their spending — with zero manual data entry required to get there.

SECTION 06

Core User Flows

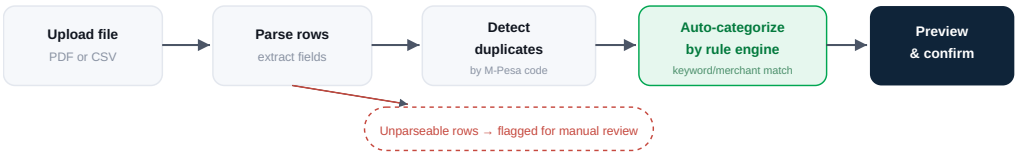
The two journeys that make or break first impressions: onboarding, and statement import.

Flow A — Onboarding to First Value



Design intent: the statement-import path is the "happy path" — it produces a populated, meaningful dashboard within under a minute, which is the core wedge described in Section 02.

Flow B — M-Pesa Statement Import (detail)



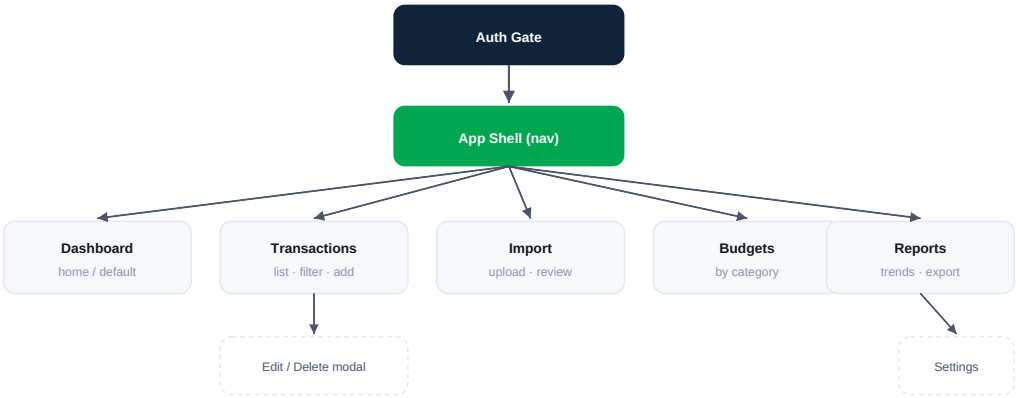
WHY THIS FLOW MATTERS MOST

Statement import is the single feature that differentiates PesaFlow from a generic expense tracker. It should get disproportionate design and QA attention relative to its share of total screens — see Section 13 for the detailed categorization-rule design.

SECTION 07

Information Architecture & Sitemap

How screens are organized and navigated in V1 — kept flat and shallow on purpose.



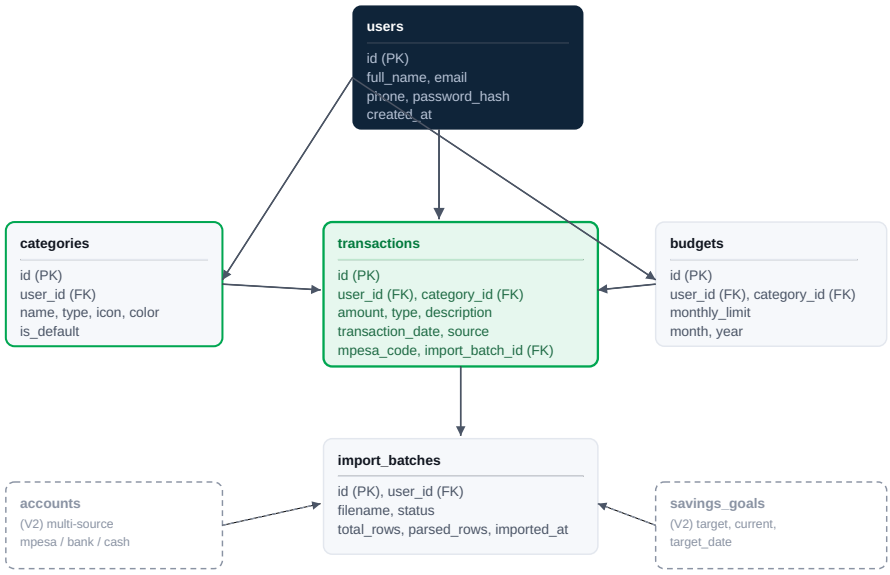
SCREEN	PRIMARY PURPOSE	KEY ACTIONS AVAILABLE
Dashboard	Orientation — "how am I doing right now"	View balance, top categories, recent activity, budget status; jump to any module
Transactions	Ground truth ledger	Search, filter, sort, add/edit/delete, paginate
Import	Bulk data entry via M-Pesa statement	Upload, preview parsed rows, fix categories, confirm import
Budgets	Intentional spending control	Set/edit monthly limits per category, view progress
Reports	Reflection & export	View monthly/trend charts, export CSV
Settings	Account housekeeping	Profile, password, theme, data reset

Deliberately flat: five primary nav items, one level of modal/detail beneath each. No nested menus, no more than two clicks from the dashboard to any action in V1.

SECTION 08

Database Design & ERD

Entity relationships for V1 (solid) with V2 extensions shown for context (dashed).



Every table carries a `user_id` foreign key from day one — even though V1 supports a single implicit account per user — so that V2's multi-account and V3's multi-tenant models extend the schema rather than migrate it from scratch.

SECTION 08 · CONTINUED

Schema Field Reference

Field-level detail for the five V1 tables.

transactions

FIELD	TYPE	NOTES
amount	Decimal	Always positive; sign implied by `type`
type	Enum	`income` / `expense`
source	Enum	`manual` / `import`
mpesa_code	String, nullable, unique per user	Used for duplicate-import detection
transaction_date	Date	Actual date of the transaction, not created_at
import_batch_id	FK, nullable	Null for manually entered transactions

categories

FIELD	TYPE	NOTES
type	Enum	Restricts category to `income` or `expense`
is_default	Boolean	Seeded system categories vs. user-created ones
icon / color	String	Drives category chips in transactions & charts

budgets

FIELD	TYPE	NOTES
monthly_limit	Decimal	One row per (user, category, month, year)
month / year	Integer	Enables historical budget comparison later

import_batches

FIELD	TYPE	NOTES
status	Enum	`pending` / `completed` / `failed`
total_rows / parsed_rows	Integer	Surfaces "12 of 14 rows imported" style feedback

MIGRATION DISCIPLINE

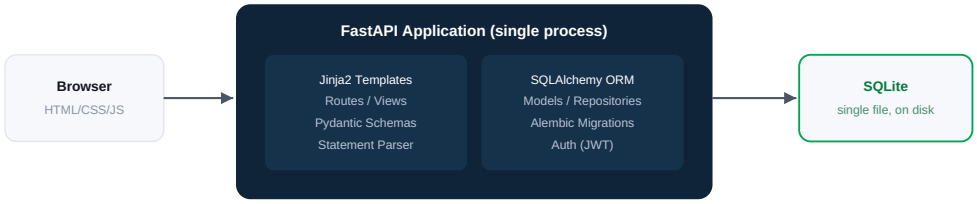
Every schema change goes through an Alembic migration from the very first commit — including the initial schema. This is the habit that makes the eventual SQLite → PostgreSQL move in V2 a non-event rather than a rewrite.

SECTION 09

System Architecture

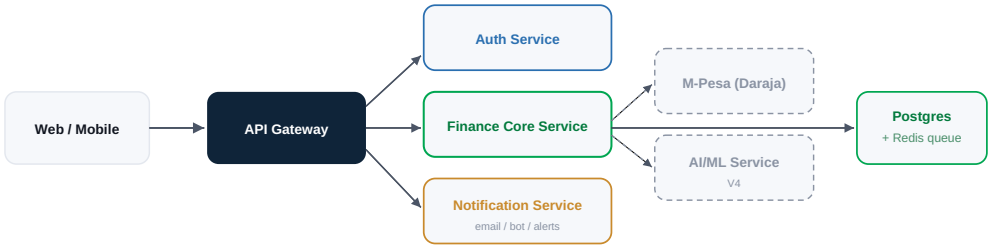
A deliberately simple V1 monolith, with a clear evolution path to services.

V1 — Monolith



One process, one deployable unit. Deployed to a single low-cost host (Render/Railway/Fly.io) with the SQLite file on a persistent volume. This is intentional — no infrastructure complexity before there are real users to justify it.

V3 — Service-Oriented Evolution (for context)



This split is shown for planning context only — it is not built until V3's multi-tenant/billing requirements actually demand independent scaling and deployment of these concerns.

SECTION 10

API Design Overview

REST resource map for V1. Kept resource-oriented so it can later be exposed as a public API in V3.

METHOD	ENDPOINT	PURPOSE
POST	/auth/register	Create a new user account
POST	/auth/login	Authenticate and issue a JWT
GET	/auth/me	Return the current authenticated user
GET / POST	/categories	List categories / create a custom category
PATCH / DELETE	/categories/{id}	Update or remove a custom category
GET / POST	/transactions	List (filtered, paginated) / create a transaction
GET / PATCH / DELETE	/transactions/{id}	Retrieve, edit, or remove a single transaction
POST	/imports/mpesa-statement	Upload a statement file, returns an import_batch preview
POST	/imports/{batch_id}/confirm	Commit a reviewed batch of parsed transactions
GET / POST	/budgets	List budgets for a month / set a new budget
PATCH / DELETE	/budgets/{id}	Update or remove a budget
GET	/reports/summary	Monthly income/expense totals
GET	/reports/by-category	Category breakdown for a given period
GET	/exports/transactions.csv	Stream a CSV export for a date range

Design conventions

CONSISTENCY RULES

- Plural nouns for collections, no verbs in URLs except for clear actions (`/confirm`)
- Pydantic schemas separate `Create`, `Update`, and `Read` shapes per resource
- All list endpoints support `page`, `page_size`, and relevant filters as query params
- All money values serialized as strings with fixed decimal precision, never floats

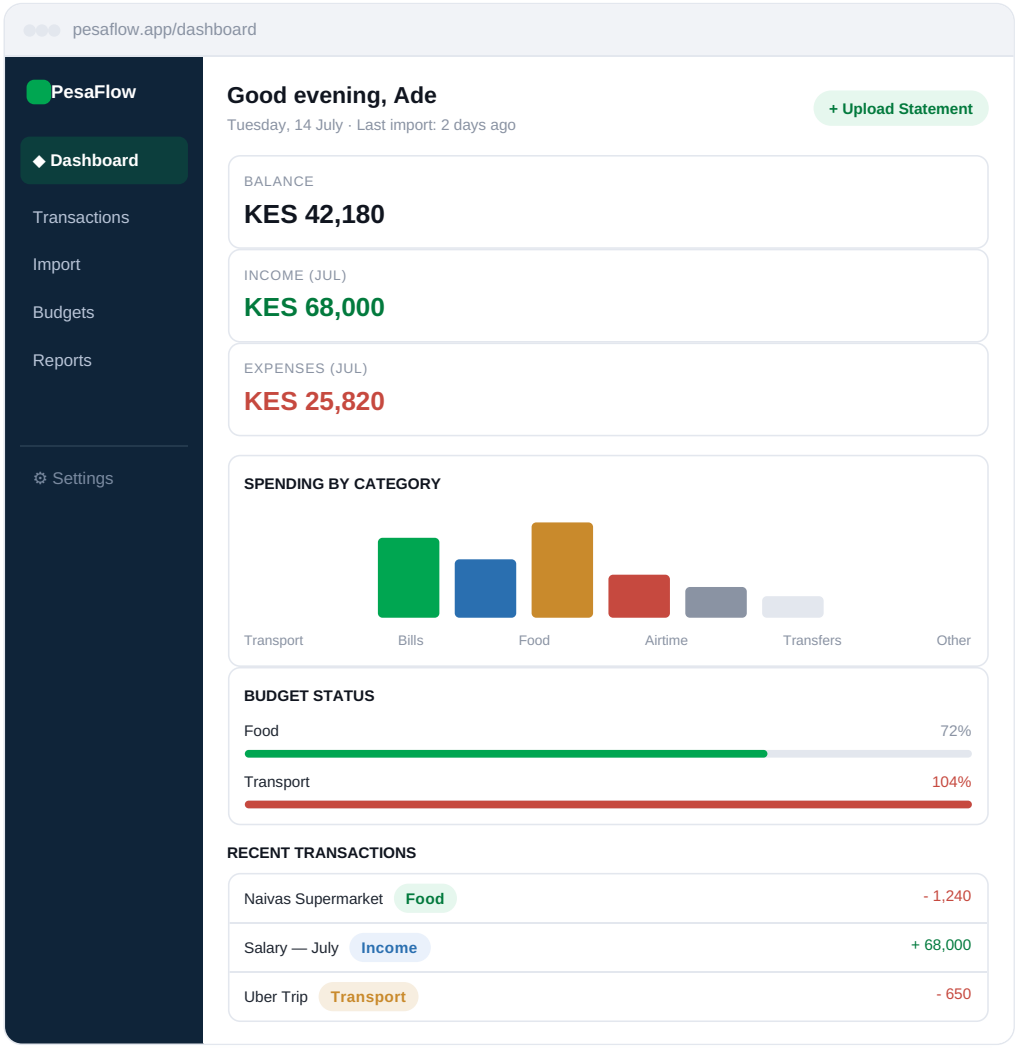
FORWARD-COMPATIBILITY NOTES

- Endpoints are user-scoped implicitly via the JWT, not via URL — this is what makes multi-account (V2) and multi-tenant (V3) additive rather than breaking
- `/imports/*` is designed so a future Daraja webhook can feed the same confirmation pipeline used by manual file upload
- Response envelopes are versioned-ready: a top-level `meta` key is reserved from V1 even if unused, to avoid a breaking change later

SECTION 11

UI/UX Concepts — Dashboard

Mid-fidelity concept for the primary landing screen after login.



Design intent: three numbers, one chart, one progress widget, three transactions — everything a user needs to answer "how am I doing" without scrolling.

SECTION 11 · CONTINUED

UI/UX Concepts — Transactions & Import Review

The two screens users will spend the most time in.

Transactions list

Search transactions | This month ▾ | All categories ▾ | + Add transaction

DATE	DESCRIPTION	CATEGORY	AMOUNT	
Jul 14	Naivas Supermarket	Food	- 1,240.00	edit
Jul 13	Kplc Postpaid	Bills	- 3,400.00	edit
Jul 12	Salary — July	Income	+ 68,000.00	edit
Jul 12	Uber Trip	Transport	- 650.00	edit
Jul 11	Airtime — Safaricom	Airtime	- 200.00	edit

‹ Page 1 of 8 ›

Import review (post-parse preview)

MPESA_Statement_June2026.pdf — 42 of 44 rows parsed

DATE	DETAIL	SUGGESTED CATEGORY	AMOUNT	
Jun 30	Customer Transfer — J. Otieno	Transfers ▾	- 2,000.00	✓
Jun 29	Pay Bill — KPLC	Bills ▾	- 3,400.00	✓
Jun 28	Till — Java House	Food ▾	- 890.00	✓
Jun 27	Unrecognized merchant code	Needs review	- 450.00	⚠

Cancel | Confirm import

Every suggested category is editable inline before confirmation — the system proposes, the user decides. This single interaction is what builds trust in the auto-categorization engine described in Section 13.

SECTION 12

Design System

A small, disciplined visual language — modern fintech, minimal, trustworthy.

Color palette

Navy — #0F2439
primary surface

Green — #00A651
brand / positive

Blue — #2A6FB0
income / info

Amber — #C98A2C
caution / bills

Red — #C6493F
expense / over-budget

Fog — #F6F8FB
neutral surface

Green is reserved for the brand mark, positive states, and primary actions — it should never be decorative. Red is reserved strictly for expenses/over-budget states so it retains meaning.

Typography scale

ROLE	SIZE / WEIGHT	USAGE
Display	28–32pt / 700	Marketing/landing headline only
Page title	18–20pt / 700	Top of each screen ("Dashboard", "Transactions")
Card title	10–11pt / 700	Widget/card headers
Body	9.5–10.5pt / 400	Default paragraph & table text
Caption / label	7–8pt / 700, uppercase, tracked	Eyebrow labels, table headers, tags
Numeric / mono	Tabular figures	All currency amounts, always right-aligned in tables

Core components

KPI CARD

Label (caption) + large number + optional trend delta. Used on Dashboard and Reports.

CATEGORY CHIP

Rounded pill, tinted background matching category color, used in tables and forms.

PROGRESS BAR

4–5px rounded track; green under budget, red at/over 100% — no intermediate warning color to keep signal binary and clear.

DATA TABLE

Uppercase tracked headers, hairline row dividers, right-aligned currency, hover row highlight in the live app.

PRIMARY BUTTON

Solid green, white text, used once per screen for the single most important action.

EMPTY / IMPORT STATE

Friendly illustration-free empty state pointing directly at "Upload your first statement."

Interaction principles

- Every destructive action (delete transaction, reset data) requires a confirm step — no silent deletes.
- Currency is always shown with two decimals and the KES prefix; never abbreviated in tables.
- Dark mode is a first-class palette swap (navy surfaces, same accent colors), not an inverted afterthought.

SECTION 13

M-Pesa Statement Import — Design Detail

The product's core differentiator deserves its own dedicated design spec.

What an M-Pesa statement contains

M-Pesa PDF/CSV statements list, per row: a transaction code, completion timestamp, a free-text "details" field (e.g. *"Customer Payment to Small Business — Java House"*, *"Pay Bill to KPLC PREPAID"*, *"Funds received from JOHN OTIENO"*), a transaction status, and paid-in / withdrawn amounts plus running balance.

Rule-based categorization engine (V1)

Rather than machine learning (deferred to V4), V1 uses a transparent, editable rule table matched against the statement's free-text detail field:

PATTERN TYPE	EXAMPLE MATCH	ASSIGNED CATEGORY
Transaction-type prefix	`"Pay Bill to KPLC*"`	Bills
Transaction-type prefix	`"Customer Payment to Small Business*"`	Shopping / Food (merchant-dependent)
Keyword in detail	`contains "UBER" / "BOLT"`	Transport
Keyword in detail	`contains "AIRTIME" / "SAFARICOM DATA"`	Airtime
Transaction-type prefix	`"Funds received from*"`	Transfers-in (income)
No rule match	–	"Needs review" (user assigns manually)

WHY RULES BEFORE ML

- Fully explainable — a user can see exactly why a transaction was tagged a certain way
- Zero training data required to launch V1
- User corrections can be captured as new personal rules immediately, improving accuracy per-user from day one

PATH TO V4 ML UPGRADE

- Every manual correction a user makes is stored — this becomes labeled training data
- Rules stay as a fast, cheap first pass; a classifier only runs on "needs review" items
- No architecture change required — categorization is already an isolated function behind one interface

Duplicate & re-import safety

Each M-Pesa transaction code is globally unique per user. On import, PesaFlow checks incoming codes against existing `transactions.mpesa_code` values for that user and silently skips exact matches — so re-uploading an overlapping statement (e.g. a new export that includes last month's tail-end) never creates duplicate entries.

SECTION 14

Non-Functional Requirements

The qualities that make PesaFlow trustworthy and pleasant to use, not just feature-complete.

QUALITY	V1 TARGET
Performance	Dashboard and transaction list render in under 1.5s on a typical mobile connection; statement import of ~100 rows parses in under 5 seconds.
Reliability	Import is transactional — a failed batch never leaves partially-inserted transactions; every write path is covered by an automated test.
Usability	A new user reaches a populated dashboard within 3 screens of signing up (see Section 06, Flow A).
Accessibility	Sufficient color contrast (WCAG AA) for all text and status colors; all interactive elements keyboard-reachable.
Maintainability	Clean layered structure (routes → services → repositories → models), consistent with your existing mid-level architecture style; every model change goes through Alembic.
Testability	Core business logic (parsing, categorization, budget calculations) is unit-tested independent of the web layer.
Portability	Config via environment variables from day one, so moving hosts or swapping SQLite → PostgreSQL never requires code changes, only configuration.

Engineering practices to carry from day one

REPOSITORY HYGIENE

- Conventional commits, feature branches, PR-style self-review even solo
- README with setup instructions, architecture diagram, and screenshots kept current
- .env.example checked in; secrets never committed

AUTOMATED CHECKS

- GitHub Actions: run tests + linting on every push
- Alembic migration check runs in CI (catches "forgot to migrate" errors early)
- Basic load/seed script to generate realistic demo data for screenshots and testing

SECTION 15

Security & Compliance

Financial data is sensitive even with a single user in a hobby-scale database.

AREA	V1 REQUIREMENT
Password storage	Bcrypt/argon2 hashing, never plaintext or reversible encryption.
Authentication	Short-lived JWT access tokens; sensitive actions (password change, data reset) re-confirm the current password.
Transport security	HTTPS-only in any deployed environment; cookies/tokens marked Secure and HttpOnly where applicable.
Input validation	All incoming data validated through Pydantic schemas; uploaded statement files size- and type-restricted before parsing.
File handling	Uploaded statements are parsed then discarded (not retained as raw files) unless the user explicitly opts to keep the source file.
Data ownership	Every query scoped strictly to the authenticated user's own records — no endpoint should ever be able to return another user's transactions.
Right to delete	A "danger zone" settings action permanently deletes a user's account and all associated data.
Rate limiting	Login and import endpoints rate-limited to blunt brute-force and abuse attempts even at small scale.

LOOKING AHEAD — V3 SAAS COMPLIANCE

Once PesaFlow accepts real paying users and potentially live M-Pesa API access, this section expands to cover: encryption at rest for PII, audit logging of every financial write, a documented data-retention policy, and — because this is financial data in Kenya — awareness of the Kenya Data Protection Act (2019) obligations around consent, storage, and breach notification.

SECTION 16

Technology Stack — Now & Future

V1 uses only tools you already know. Everything else is queued, not required.

LAYER	V1 (NOW)	V2 – V4 (FUTURE)
Language	Python (intermediate)	Same — depth increases, not breadth
Backend framework	FastAPI	FastAPI, split into services in V3
ORM / migrations	SQLAlchemy + Alembic	Same, against PostgreSQL
Validation	Pydantic	Same, plus request-signing for public API (V3)
Frontend	Jinja2 templates + vanilla JS + CSS	Progressive enhancement → PWA (V2); possible React/mobile client (V3)
Database	SQLite	PostgreSQL (V2) + Redis cache/queue (V3)
Background jobs	None (synchronous)	APScheduler → Celery/RQ (V2–V3)
Auth	JWT, self-managed	Same, extended with refresh-token rotation
Notifications	None	Email (V2), Telegram Bot API (V3)
Payments	None	Stripe/Paystack/M-Pesa STK Push (V3)
External APIs	None (file-based import)	Safaricom Daraja API (V3)
AI / ML	None (rule-based)	LangChain + RAG, vector DB / pgvector (V4)
Infra / deploy	Single host (Render/Railway/Fly.io), Git-based deploy	Docker, CI/CD pipeline, possible container orchestration at real scale
Tooling	Git, GitHub, VS Code	Same, plus GitHub Actions (CI), Sentry-style error monitoring (V2+)

SKILL-GROWTH ALIGNMENT

This progression doubles as a learning roadmap toward your stated AI-engineering and bot-development goals: PostgreSQL and background jobs in V2 build production backend maturity; Telegram Bot API and webhooks in V3 directly serve your bot-development interest; LangChain/RAG in V4 is your entry point into applied AI engineering — all learned in the context of a product you already understand deeply.

SECTION 17

Success Metrics & KPIs

How you'll know whether V1 is actually working, beyond "it's deployed."

METRIC	V1 TARGET	WHY IT MATTERS
Time to first populated dashboard	< 60 seconds after statement upload	Validates the core wedge — value without manual data entry
Statement parse success rate	> 90% of rows categorized without manual review	Directly measures the rule engine's real-world accuracy
Weekly active use (self)	Opened & used ≥ 4 days/week for 4 consecutive weeks	The strongest signal a solo project is worth continuing to invest in
Budget adherence rate	Tracked, not targeted, in V1	Establishes a baseline before V2 introduces alerts/nudges
Manual-review backlog	< 10% of imported transactions left "needs review" after 7 days	Signals whether the rule table needs expansion
Portfolio outcome	Live demo + README + architecture diagram ready to show	V1's real "customer" is often a recruiter or collaborator

MEASUREMENT DISCIPLINE

You don't need an analytics platform for V1 — a simple `events` log table (user_id, event_name, metadata, timestamp) written to on key actions (import completed, budget created, dashboard viewed) is enough to compute every metric above with a SQL query.

SECTION 18

Risks & Mitigations

Named honestly, planned for early — the mark of a real product plan.

RISK	IMPACT IF IGNORED	MITIGATION
M-Pesa statement format varies or changes over time	Parser silently breaks or miscategorizes data	Build the parser around a versioned template/schema; log and surface parse failures instead of failing silently; keep a small fixture library of real (anonymized) statement formats for regression testing
Scope creep — chasing V2/V3 ideas before V1 ships	V1 never actually ships; motivation stalls	Freeze the V1 feature list in Section 05; log every new idea into the roadmap backlog instead of the current sprint
Handling sensitive financial data casually	Trust is broken instantly if data leaks or is mishandled, even for one user	Follow Section 15 from commit one — security is not a "later" feature
Solo-developer bandwidth	Burnout, abandoned project	Phase gating (Section 04); each phase has a hard "done" definition before the next begins; timebox V1 to a fixed number of weeks
Low real-world adoption beyond yourself	Feels like the project "failed"	Define success partly as a portfolio/learning outcome (Section 17) so the project has value independent of user growth
Over-engineering V1 architecture "for scale"	Slower V1 delivery for benefits that don't matter yet	Follow the monolith-first architecture in Section 09; introduce services only when V3 requirements concretely demand them

SECTION 19

Development Checklists

Three checkpoints: before you write code, while building V1, and before you call it launched.

PRE-DEVELOPMENT SETUP

- ☐ Repo initialized with README skeleton, .gitignore, .env.example
- ☐ Finalize V1 schema against Section 08 ERD
- ☐ Alembic configured with initial migration
- ☐ Base FastAPI app with health-check route deployed to chosen host
- ☐ CI workflow (GitHub Actions) running lint + tests on push
- ☐ Collect 2–3 real (anonymized) M-Pesa statements for parser testing

V1 BUILD CHECKLIST

- ☐ Auth: register, login, JWT issuance, protected routes
- ☐ Categories: seed defaults, CRUD for custom categories
- ☐ Transactions: full CRUD, filtering, pagination
- ☐ Statement parser: PDF/CSV → structured rows, with tests against real samples
- ☐ Categorization rule engine + "needs review" fallback
- ☐ Import preview/confirm flow with duplicate detection
- ☐ Budgets: CRUD + progress calculation
- ☐ Dashboard: balance, income/expense, category chart, budget widget, recent activity
- ☐ Reports: monthly summary, category breakdown, trend view
- ☐ CSV export
- ☐ Settings: profile edit, password change, theme toggle, data reset

LAUNCH CHECKLIST

- ☐ README with setup steps, screenshots, architecture diagram
- ☐ Seed script producing realistic demo data
- ☐ Live deployed demo URL with a safe demo login (or self-signup)
- ☐ Error states designed (failed import, empty states, validation errors)
- ☐ Basic test suite passing in CI
- ☐ Short demo video or GIF walkthrough for portfolio use
- ☐ Portfolio write-up: problem, decisions, trade-offs, what you'd do next
- ☐ Roadmap (Section 04) published in the repo so reviewers see the trajectory

SECTION 20

Closing Notes & Next Steps

Where this document ends and the build begins.

This blueprint deliberately over-plans relative to what V1 needs, on purpose: a clear map of V2–V4 means every decision you make today can be evaluated against where the product is actually headed, not just what's convenient this week. But the discipline that makes this work is sequencing — **build V1 completely before opening any V2 door.**

THIS WEEK

Finalize the schema, stand up the FastAPI skeleton, and get one real M-Pesa statement parsing into raw structured rows.

THIS MONTH

Work through the V1 build checklist (Section 19) top to bottom; resist adding anything from Section 04's later phases.

BEFORE CALLING V1 "DONE"

Use PesaFlow yourself for at least two full weeks on real data before deciding what V2 should actually contain.

PesaFlow's advantage was never going to be technical complexity — it's specificity. A tool built specifically around how money actually moves for its target users, built by someone who is one of those users. Keep that specificity as the product grows; it's the thing generic finance apps can't copy.